

Critical Flows — Chris Stipes Portfolio Site

For coding agents: read this before changing anything under `app/` , `lib/` , or `tests/` . Each flow lists invariants that other code and tests depend on. If your change alters an invariant, update this document in the same change and re-run the smoke suite.

Generated	2026-07-30
Commit	c091f25
Verified against	https://www.stipes.tech on 2026-07-31 (all 5 smoke flows passed)
Regenerate with	/document-flows [url]

Deployment note: `stipes.tech` 307-redirects to `www.stipes.tech` . Smoke runs pass against either, but prefer the canonical `www` host to avoid an extra redirect on every navigation.

Flow summary

#	Flow	Criticality	Entry point	External deps
1	Chat assistant	P0	/ (widget)	stipes-openai-chat.vercel.app → OpenAI
2	Home page render	P0	/	none
3	Blog index → post	P1	/blog , /blog/[slug]	none (filesystem MDX) ⚠ see defect
4	Resume download	P1	/resume.pdf	none (static asset)
5	About page	P2	/about	none

How to use this document

- **Before editing** a file listed under any flow, read that flow's Invariants.
- **When an invariant must change**, change it deliberately: update the invariant here, update the covering tests, and note it in the commit.
- **After deploying**, run `SMOKE_BASE_URL=https://<deployment> npm run smoke` .
- **Criticality:** P0 breaks the site for every visitor or depends on an external service; P1 is significant but self-contained; P2 is peripheral.

Flow 1: Chat assistant

Criticality: P0 **Verified:** source-only

What the user does

A floating "Ask a Stipes bot!" button sits in the bottom-right corner of the home page. Clicking it opens a chat panel seeded with an assistant greeting. The user types a question, presses Send, and an answer appears as a message bubble. The panel can be minimized back to the button.

Implementation chain

1. `app/(site)/page.tsx` — mounts `<ChatWidget />`
2. `app/(site)/components/ChatWidget.tsx` — `"use client"`; open state,

closed by default; renders the launcher button or the panel + header

1. `app/(site)/components/ChatWindow.tsx` — `"use client"`; message list, input form, `handleSend` performs the `fetch`

1. `app/api/chat/route.ts` — Next.js route handler; validates and proxies
2. External Go service — wraps the OpenAI API

Network and external dependencies

- `POST /api/chat` with body `{ message: string }` → `app/api/chat/route.ts`
- That handler forwards to `process.env.CHAT_API_URL`, defaulting to `https://stipes-openai-chat.vercel.app/chat` (`app/api/chat/route.ts:5,15`)
- The upstream is a separate deploy (`github.com/cstipes-devs/stipes-openai-chat`)

and can fail independently of this site — this is why the flow is P0.

Invariants — do not break these

- **The launcher button keeps `aria-label="Open Stipes bot"`**

(`ChatWidget.tsx:14`). Both `tests/e2e/chat.spec.ts` and the smoke suite select on it; there is no `data-testid` fallback anywhere in this repo.

- **The input keeps `placeholder="Type your message..."`**

(`ChatWindow.tsx:122`) and the submit button keeps its accessible name `Send` (`ChatWindow.tsx:130`). Both are selected on by tests. Note the button label switches to `Sending...` while a request is in flight, so selectors must match `Send` exactly rather than by prefix.

- **The request body stays `{ message: prompt }`** (`ChatWindow.tsx:47`). The

route handler accepts `body.message ?? body.prompt` (`app/api/chat/route.ts:10`), but the upstream Go service expects `message` .

- **Do not narrow the response-shape fallback chain** in `ChatWindow.tsx:62-73` :

`choices[0].message.content` → `message.content` → `answer` → `reply` → `response` → `message` → `text` → `output` → `JSON.stringify(data)` . The upstream contract is loose and has returned more than one of these shapes; collapsing the chain to a single field silently produces blank replies.

- **Blank or missing messages must return 400** (`app/api/chat/route.ts:11-13`).

The client guards on `canSend` too, but the server check is the real contract.

- **Non-JSON upstream responses must still render.** The route passes through

text bodies with their content type (`route.ts:37-41`) and the client reads `res.text()` when the type is not JSON (`ChatWindow.tsx:74-76`).

- **Upstream errors surface as a visible red error bubble**, not a silent

failure — `role: "error"` messages (`ChatWindow.tsx:80-84` , rendered at `ChatWindow.tsx:107`). Smoke tests assert the reply is *not* one of these.

- **CHAT_API_URL must keep a working default.** The site is deployed without

this variable set; removing the literal default breaks chat in production.

Covering tests

Test	File	What it certifies
response-shape contract	tests/e2e/chat-errors.spec.ts	Every shape in the fallback chain renders (one test per shape), plus raw-JSON last resort and plain-text passthrough (mocked via the <code>chatApi</code> fixture)
error handling	tests/e2e/chat-errors.spec.ts	Upstream 500 → error bubble + recovery; network abort → error bubble; empty/whitespace input cannot send (mocked)
easter egg reply	tests/e2e/chat.spec.ts	Widget opens, sends, renders a reply (mocked via <code>chatApi.onlyFor</code>)
open/minimize	tests/e2e/chat.spec.ts	Panel toggling through the accessible buttons
widget unit tests	tests/unit/chatWidget.test.tsx	Open/minimize toggle, accessible labels
window unit tests	tests/unit/chatWindow.test.tsx	Message rendering, send behavior, response parsing
live chat round trip	tests/e2e/smock.spec.ts	Real <code>/api/chat</code> → real upstream returns a non-error reply (never mocked)

All browser selectors for this flow live in `tests/e2e/pages/ChatWidget.ts`. Accessibility gap worth fixing: the message list and the error state expose no role or `aria-live` region — error bubbles are distinguishable only by color (`bg-red-900/50`), forcing the page object's one class-based locator.

Failure modes

Chat button missing → `ChatWidget` failed to mount; check `page.tsx`. Red error bubble → upstream is down or `CHAT_API_URL` is misconfigured; check the Go service deploy. Reply renders as raw JSON → the upstream changed its response shape and none of the fallback keys matched.

Flow 2: Home page render

Criticality: P0 **Verified:** source-only

What the user does

Landing on `/` shows the sticky navbar, a hero with photo, name, tagline, and two calls to action, followed by stat cards, capability sections, a featured blog card, and the footer.

Implementation chain

1. `app/layout.tsx` — root shell and site metadata
2. `app/(site)/page.tsx` — async server component; awaits `getAllPostMetadata()`

for the featured blog card

1. `app/(site)/components/` — `Navbar`, `Hero`, `AngledDivider`, `StatCard`, `Section`, `WipIcon`, `Footer`, `ChatWidget`

Network and external dependencies

None at request time. The page is a server component reading MDX frontmatter from the local filesystem.

Invariants — do not break these

- **The `<h1>` stays "Chris Stipes"** (`Hero.tsx`). `tests/e2e/home.spec.ts`

selects `getByRole('heading', { level: 1, name: /chris stipes/i })`, and it must remain the *only* h1 on the page.

- **`page.tsx` stays an async server component.** It calls

`getAllPostMetadata()`, which uses `node:fs` and cannot run in a client component. Adding `"use client"` here breaks the build.

- **`posts.length > 0` and `posts.length < 2` guards stay**

(`page.tsx`, Blog section). With zero posts the featured card must not render and must not throw on `posts[0]`.

- **The hero image keeps `priority`** (`Hero.tsx`) — it is the LCP element.
- **The featured blog card's heading and description are hardcoded**

(`page.tsx`, Blog section) rather than read from frontmatter. It therefore still displays correctly despite the Flow 3 frontmatter defect — but it will not track a post's real title if one is ever set. Only `posts[0].slug` and `posts[0].frontmatter.badge` come from the MDX.

- **Anchor targets `#top`, `#work`, and `#blog` must exist.** The navbar and

in-page links depend on them; `#blog` is a bare `<div id="blog" />`.

Covering tests

Test	File	What it certifies
home renders	<code>tests/e2e/home.spec.ts</code>	The h1 is visible (via <code>HomePage</code> page object)
component units	<code>tests/unit/hero.test.tsx</code> , <code>navbar.test.tsx</code> , <code>statCard.test.tsx</code> , <code>footer.test.tsx</code> , <code>section.test.tsx</code> , <code>angledDivider.test.tsx</code> , <code>wipIcon.test.tsx</code>	Individual render contracts
home smoke	<code>tests/e2e/smoke.spec.ts</code>	h1 and chat launcher both present on the deployment

Selectors for this flow live in `tests/e2e/pages/HomePage.ts`.

Failure modes

Blank page or 500 → most often `getAllPostMetadata()` throwing because `content/posts/` is missing from the deployment bundle.

Flow 3: Blog index → post

Criticality: P1 **Verified:** source-only

What the user does

`/blog` lists every post with badge, title, description, date, and reading time. Clicking a title opens `/blog/<slug>`, which renders the full MDX article with a formatted date header.

Implementation chain

- `app/(site)/blog/page.tsx` — async server component; `getAllPostMetadata()`
- `app/(site)/blog/[slug]/page.tsx` — `generateStaticParams()`, `generateMetadata()`, `getPost(params.slug)`, `notFound()` on miss
- `lib/mdx.ts` — reads `content/posts/<slug>.mdx` from disk; `compileMDX` with `parseFrontmatter: true` and `remarkGfm`; `gray-matter` for metadata
- `content/posts/*.mdx` — currently one post, `chat-bot.mdx`

Network and external dependencies

None. Content is read from the filesystem at build time.

Invariants — do not break these

- **content/posts/ must ship with the deployment.** `POSTS_DIR` is resolved

from `process.cwd()` (`lib/mdx.ts:16`); if the directory is absent, both blog routes throw and the home page fails with them.

- **A missing slug returns 404, never a crash.** `getPost` catches `ENOENT` and

returns `null` (`lib/mdx.ts:22–27`), and the page calls `notFound()` (`[slug]/page.tsx:30–32`). Keep both halves.

- **Frontmatter requires `title`, `description`, `publishedAt`;** `badge` and

`readingTime` are optional and have rendered fallbacks (`?? "Case Study"`, conditional reading time). Do not make optional fields required.

- **Invalid `publishedAt` must degrade, not crash.** Both pages check

`Number.isNaN(date.getTime())` and fall back to the raw string.

- **`getAllPostMetadata()` sorts descending by `publishedAt`**

(`lib/mdx.ts:76–78`) — newest first is the expected index order.

- **`remarkGfm` stays enabled** (`lib/mdx.ts:41`). Existing posts use GFM

tables and strikethrough; removing it silently mangles them.

- **Every post file must open with a `---` frontmatter block.** Without one,

`gray-matter` returns empty data and the index renders titleless, dateless, empty links — see the defect below.

⚠ Known defect — CONFIRMED LIVE IN PRODUCTION (2026-07-31):

`https://www.stipes.tech/blog/chat-bot` renders **no `<title>` tag** and **two `<h1>` elements**; the `/blog` index post link has empty text. `content/posts/chat-bot.mdx` **has no frontmatter block at all** — the file begins directly with body content. It has never had one (`git show fba18c1:content/posts/chat-bot.mdx`). Consequences: - `/blog` renders the post as an **empty, invisible link**: no title, no description, no date. Only the `badge` renders, via its `?? "Case Study"` fallback. - `/blog/chat-bot` renders an empty `<h1>` and no date. - `generateMetadata()` returns an empty title and description, so the post has no meaningful SEO metadata. The route itself resolves and the article body renders, which is why `tests/e2e/blog.spec.ts` (asserting only `response.ok()`) has always passed. **Fix:** prepend a frontmatter block to `content/posts/chat-bot.mdx`: ``yaml --- title: "Make your portfolio stand out with AI!" description: "A minimal Go service wrapping OpenAI plus a Next.js/Tailwind frontend with a wired chat widget." publishedAt: "2025-09-30" badge: "Case Study" readingTime: "8 min read" ---`` Once fixed, tighten the blog smoke assertion from `toBeAttached()` back to `toBeVisible()` and navigate by clicking the link instead of following its `href`` directly.

Covering tests

Test	File	What it certifies
post loads	<code>tests/e2e/blog.spec.ts</code>	<code>/blog/chat-bot</code> responds OK and renders an article
missing post 404s	<code>tests/e2e/blog.spec.ts</code>	Unknown slug returns 404, not a crash
mdx units	<code>tests/unit/mdx.test.ts</code>	Frontmatter parsing, sorting, missing-slug null
blog smoke	<code>tests/e2e/smoke.spec.ts</code>	Index lists posts and navigation reaches a rendered post

Selectors live in `tests/e2e/pages/BlogIndexPage.ts` and `BlogPostPage.ts`.

⚠ Drift (live-verified): the post page renders **two** `<h1>` elements — the frontmatter title (currently empty, see the defect above) and a second `<h1>` from the MDX body's own `#` heading in `chat-bot.mdx`. Duplicate h1s are an accessibility/SEO problem independent of the missing frontmatter; when fixing the frontmatter, also demote the body heading to `##`.

Failure modes

404 on a post that exists → slug/filename mismatch. Index empty → `POSTS_DIR` not resolving in the deployment.

Flow 4: Resume download

Criticality: P1 **Verified:** source-only

What the user does

Clicks "Download Resume (PDF)" in the navbar or the hero and gets the resume PDF.

Implementation chain

1. `app/(site)/components/Navbar.tsx` — links the current resume asset
(`/resume072026.pdf` as of 2026-07-30)
1. `app/(site)/components/Hero.tsx` — links `/ChristopherStipesResume_v3.pdf`
2. `public/resume072026.pdf` — the static asset; the filename is dated and renamed periodically when the resume is updated

Network and external dependencies

None; a static file served from `public/`.

Invariants — do not break these

- Every resume link must point at a file that exists in `public/`. The

asset filename is dated and changes on each resume update; renaming it without updating every referring link produces a 404. The smoke test follows the navbar link's actual `href` (rename-proof), but `tests/unit/navbar.test.tsx` asserts the literal path — update it in the same change as any rename.

⚠ Known defect — CONFIRMED LIVE IN PRODUCTION (2026-07-31):

`https://www.stipes.tech/ChristopherStipesResume_v3.pdf` returns **404**, while `/resume072026.pdf` returns 200. `Hero.tsx` links `/ChristopherStipesResume_v3.pdf`, which does not exist in `public/`. **The hero's "Download Resume (PDF)" button 404s today.** The navbar link is correct. Fix by pointing the hero at the current asset (`/resume072026.pdf`) — or better, have both components share one constant so future renames touch a single place. The smoke suite certifies the navbar link only — deliberately, so this doc records the defect rather than a test masking it.

Covering tests

Test	File	What it certifies
resume asset	<code>tests/e2e/smoke.spec.ts</code>	The navbar link's href returns 200 with a PDF content type
navbar links	<code>tests/unit/navbar.test.tsx</code>	Navbar hrefs, including the literal resume path

The navbar's resume link locator lives in `tests/e2e/pages/HomePage.ts` (`navResumeLink`).

Failure modes

404 on download → asset missing from the deployment or a link path drifted from the filename.

Flow 5: About page

Criticality: P2 **Verified:** source-only

What the user does

Visits `/about` for a static résumé: contact details and experience entries.

Implementation chain

1. `app/(site)/about/page.tsx` — fully static server component

Network and external dependencies

None.

Invariants — do not break these

- `/about` must return 200. The navbar links to it from every page.
- **No data fetching belongs here.** The page is intentionally static; adding

a loader gives it a new failure mode it does not currently have.

Covering tests

Test	File	What it certifies
about reachable	<code>tests/e2e/smoke.spec.ts</code>	<code>/about</code> responds OK and renders a heading (via <code>tests/e2e/pages/AboutPage.ts</code>)

Failure modes

404 → route removed or renamed while the navbar link stayed.

Test architecture

For the cross-repo picture — how this suite relates to the Go backend's four test layers, the wire contract between them, and known gaps — see [TEST_ARCHITECTURE.md](#).

The Playwright suite is built on page objects and typed fixtures (regenerate with `/playwright-tests`):

- `tests/e2e/pages/` — page/component objects. ****The only place selectors**

live.** Every locator traces to an invariant in this document.

- `tests/e2e/fixtures.ts` — `test.extend` providing one fixture per page

object plus `chatApi`, the lazy `/api/chat` mock (`replyWith`, `failUpstream`, `abort`, `onlyFor`, ...). All specs import `test / expect` from here, never from `@playwright/test`.

- Tiers: local e2e (`home/blog/chat.spec.ts`, mocking allowed via `chatApi`

only), error contract (`chat-errors.spec.ts`, fully mocked — the negative space of the loose upstream contract), smoke (`smoke.spec.ts`, `@smoke`, never mocked, never writes).

Running the smoke suite

```
# Against a deployment (no local server is started)
SMOKE_BASE_URL=https://your-deployment.vercel.app npm run smoke

# Against a local server (auto-started by Playwright)
npm run smoke:local
```

`playwright.config.ts` switches on `SMOKE_BASE_URL` : when set, it becomes the `baseUrl` , `webServer` is disabled, the timeout rises to 60s, failed tests retry twice to absorb network flake, and **the run is filtered to @smoke specs only** — the other tiers mock the network, so pointing them at a real deployment is meaningless.

`SMOKE_BASE_URL` may be set persistently in `.env.local` (loaded via `@next/env`); the `@smoke` filter above is what makes that safe. To force a full local run while it is set, pass an empty value: `SMOKE_BASE_URL= npm run e2e` .

`PLAYWRIGHT_BASE_URL` is a different lever: it changes the **local** origin (and the port the dev server is started on) for when `:3000` is taken. It does not disable `webServer` — never point it at a deployed site.

Smoke tests never mock network traffic — mocking the dependency under certification would defeat their purpose — and never perform writes against a real deployment.

Notes

Content between the manual markers is preserved verbatim when this document is regenerated. Put human-authored context here.